

- RAG Learning

1.1 Characteristics of Contemporary Processors, Input, Output and Storage Devices

Computer Science — OCR — Component 01 (Computer Systems)

1.1.1 Structure and Function of the Processor

Von Neumann vs Harvard Architecture

The **Von Neumann architecture** stores both **program instructions and data in the same memory**, accessed through a **single shared bus**. Instructions are fetched and executed **sequentially** using the stored program concept (1945). Virtually all general-purpose computers (PCs, servers, smartphones running application code) use this model.

The **Harvard architecture** uses **physically separate memories** for instructions and data, connected by **separate buses**. This allows the CPU to fetch an instruction and read data **simultaneously**, improving throughput. It is common in embedded systems, microcontrollers (PIC, AVR, ARM Cortex-M) and digital signal processors (DSPs).

Modern high-performance CPUs (x86, ARM Cortex-A) are said to be **modified Harvard**: the main memory is unified (Von Neumann), but the on-chip L1 cache is split into a separate **instruction cache (L1i)** and **data cache (L1d)**. This gives Harvard-style simultaneous fetch at cache level while retaining Von Neumann's programming flexibility.

Feature	Von Neumann	Harvard
Memory	Single unified memory for instructions + data	Separate instruction memory and data memory

Buses	One shared bus (address + data + control)	Separate buses for instructions and data
Simultaneous fetch/read?	No — one at a time (Von Neumann bottleneck)	Yes — instruction and data accessed in parallel
Flexibility	High — instructions and data interchangeable (self-modifying code possible)	Lower — instruction memory often read-only
Complexity / cost	Simpler, cheaper hardware	More complex, higher hardware cost
Typical use	General-purpose computers (desktops, laptops, servers)	Embedded systems, microcontrollers, DSPs (audio, radio, motor control)
Example	x86 PC running Windows	ATmega328 (Arduino Uno), TI C6000 DSP

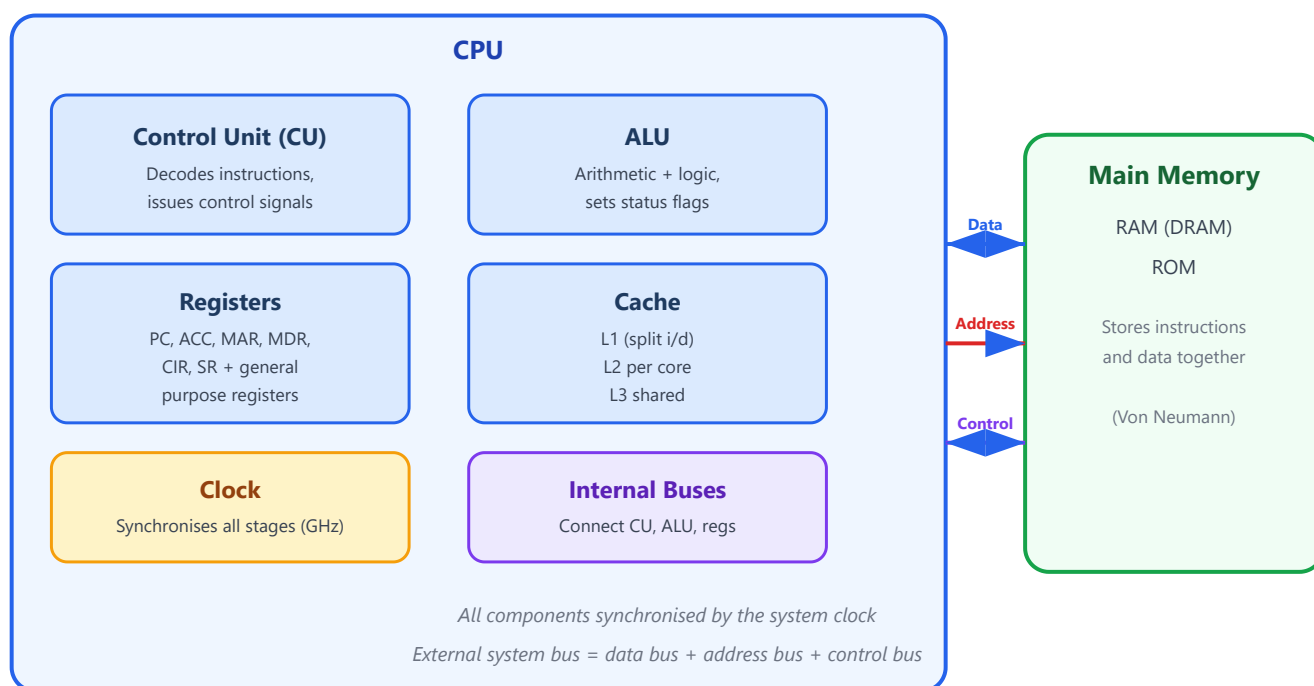
Exam Tip: The **Von Neumann bottleneck** is the fact that instructions and data share a single bus, so only one can be transferred at a time. Mentioning this by name earns marks in compare/contrast questions. Harvard architecture removes this bottleneck by duplicating the bus.

CPU Internal Structure

The CPU contains the ALU, Control Unit, registers, cache, and an internal clock, all connected by internal buses. These communicate with main memory and I/O via the external system bus.

At A-Level it is essential to understand not just what each component does, but *why* each component exists and how they co-operate. The ALU performs the arithmetic and logic, but it needs operands to be placed where it can access them (registers) and a sequence of steps to be coordinated (CU, clock). The CU sequences those steps by issuing control signals, but it needs to know what to sequence (CIR) and where to find the next step (PC). Each component is therefore only useful in the context of the others — this tightly coupled interaction is what makes a processor a processor rather than a collection of parts.

CPU Internal Structure



Key CPU Components

Component	Function	A-Level detail
ALU (Arithmetic Logic Unit)	Performs all arithmetic and logical operations	Arithmetic (ADD, SUB, MUL, DIV, shifts), logic (AND, OR, NOT, XOR) and comparisons. Reads operands from registers, writes result back to a register (typically ACC) and updates status register flags (zero, negative, carry, overflow).
Control Unit (CU)	Orchestrates execution of instructions	Fetches instructions, decodes the opcode, and issues micro-operations / control signals on the control bus to direct data flow between registers, ALU, cache and memory. Can be hardwired (fast, fixed logic) or microcoded (flexible, uses a small program in ROM).

Registers	Tiny, extremely fast on-die storage	Accessed in a single clock cycle. Include dedicated special-purpose registers (PC, MAR, MDR, CIR, ACC, SR) plus a bank of general-purpose registers (e.g. R0–R15 on ARM, RAX/RBX/... on x86-64) used to hold operands and intermediate results.
Cache	Fast SRAM buffer between CPU and RAM	Keeps copies of recently used instructions/data. Organised as L1 (split i/d, per-core), L2 (per-core), L3 (shared). Reduces average memory access time by exploiting locality of reference .
Clock	Synchronises operations	Crystal oscillator produces a square-wave signal at a fixed frequency (e.g. 3.5 GHz = 3.5×10^9 cycles/sec). Each micro-operation takes one or more clock cycles.
Buses	Communication pathways	See next section — address, data and control buses.

Registers in Detail

A-Level requires you to know **five special-purpose registers** (PC, MAR, MDR, CIR, ACC); the status register (SR) and its flags are useful extra context.

Register	Name	Holds	Role
PC	Program Counter	Address	Address of the <i>next</i> instruction to fetch. Incremented each fetch; overwritten by branch instructions.
MAR	Memory Address Register	Address	Holds the address currently being placed on the address bus for a read or write.
MDR	Memory Data Register	Data/Instruction	Buffer for the data going to/returning from memory on the data bus. Also called MBR.
CIR	Current Instruction Register	Instruction	Holds the instruction currently being decoded and executed. Split into opcode +

			operand by the CU.
ACC	Accumulator	Data	Default destination for ALU results; primary working register in LMC-style architectures.
SR	Status Register (flags register)	Bit flags	Individual bits set by the ALU to record properties of the last operation. Drives conditional branches.

Typical Status Register Flags

Flag	Set when...	Used by
Z (Zero)	Result of last operation is zero	BEQ / BNE (branch if equal/not-equal to zero)
N (Negative / Sign)	Result has MSB = 1 (negative in two's complement)	BMI / BPL (branch if minus / plus)
C (Carry)	Unsigned arithmetic generated a carry/borrow out of MSB	Multi-word arithmetic, unsigned comparisons
V (Overflow)	Signed arithmetic overflowed the representable range	Signed comparisons, arithmetic error detection
I (Interrupt enable)	Interrupts currently enabled	OS interrupt handling

Exam Tip: Common exam mistake: confusing **MAR** and **MDR**. Memorise: **MAR = Memory Address Register (addresses, connects to address bus)**; **MDR = Memory Data Register (data/instructions, connects to data bus)**. Another common error is forgetting the PC is incremented *during* fetch (not after execute). State $PC \leftarrow PC + 1$ as the second step.

Buses — Function and Typical Widths

Bus	Carries	Direction	Typical width	Effect of wider bus
-----	---------	-----------	---------------	---------------------

Address Bus	Memory addresses specifying the location to read/write	Unidirectional (CPU → memory)	32 or 64 bits (some embedded: 16–24)	More addressable memory: n -bit bus → 2^n locations. 32-bit = 4 GiB; 64-bit = 16 EiB (theoretical).
Data Bus	Actual data and instructions being transferred	Bidirectional	64 bits (modern desktop); 128/256/512 in memory channels	More data per transfer → higher throughput, fewer memory cycles for wide values.
Control Bus	Control signals: READ , WRITE , clock, interrupt request (IRQ), bus request/grant, reset	Bidirectional (mix of CPU-out and device-in signals)	Typically ~10–20 individual lines	More lines = more control signals → richer behaviour (e.g. DMA, multiple IRQs).

Bus width = the number of parallel wires in the bus, i.e. how many bits can be transmitted simultaneously. The **address bus width** determines the **maximum addressable memory**; the **data bus width** determines **how much data is transferred per cycle**.

The Fetch-Decode-Execute (FDE) Cycle

The FDE cycle is the fundamental execution loop of a Von Neumann CPU. At A-Level you must be able to describe each step using **register transfer notation (RTN)**, e.g. `MAR ← [PC]` means "the contents of PC are copied into MAR".

Fetch

- | | | |
|--------|-----------------|--|
| 1. MAR | ← [PC] | ; address of next instruction into MAR |
| 2. PC | ← [PC] + 1 | ; increment PC (ready for next fetch) |
| 3. MDR | ← [Memory[MAR]] | ; CU issues READ; data bus returns value |
| 4. CIR | ← [MDR] | ; copy instruction into CIR, freeing MDR |

Decode

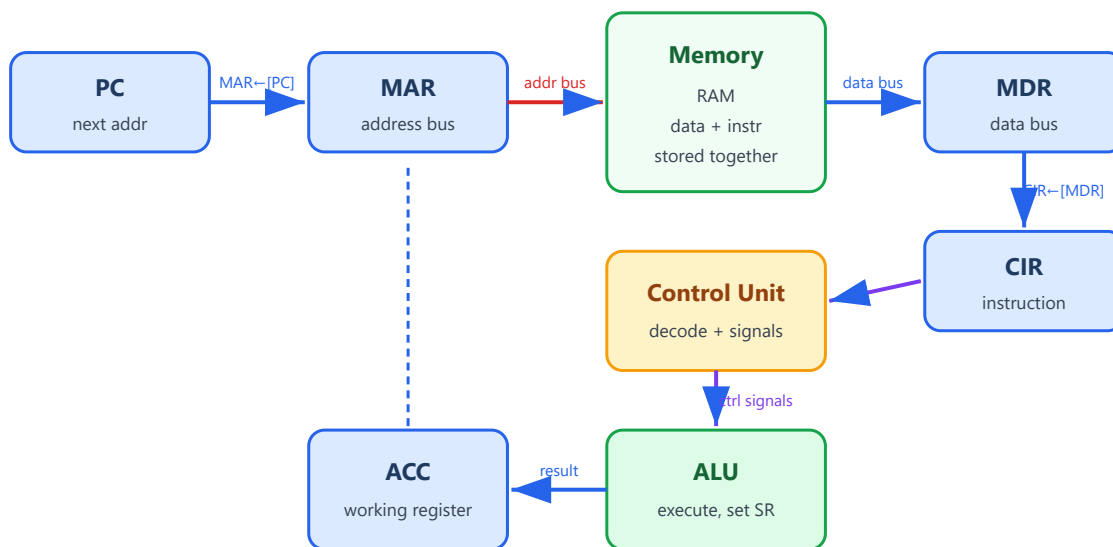
5. CU splits [CIR] into opcode and operand
6. CU selects the appropriate micro-operations for this opcode

Execute

- 7a. (Arithmetic) $ACC \leftarrow [ACC] \text{ OP } [operand]$; ALU sets SR flags
- 7b. (Load) $MAR \leftarrow operand$
 $MDR \leftarrow [Memory[MAR]]$
 $ACC \leftarrow [MDR]$
- 7c. (Store) $MAR \leftarrow operand$
 $MDR \leftarrow [ACC]$
 $Memory[MAR] \leftarrow [MDR]$; CU issues WRITE
- 7d. (Branch taken) $PC \leftarrow operand$; changes next fetch address

After the execute phase the cycle repeats — step 1 begins again using the new PC.

Fetch-Decode-Execute Cycle — Data Flow



Exam Tip: At A-Level you are expected to use **register transfer notation**. Write $MAR \leftarrow [PC]$ rather than prose. Square brackets mean "the contents of"; the arrow means "copy into". Always show the PC increment on its own line: $PC \leftarrow [PC] + 1$.

Worked FDE Trace — Three Instructions

Memory layout:

```

0:  LOAD  10      (load value at addr 10 into ACC)
1:  ADD   11      (add value at addr 11 to ACC)
2:  STORE 12      (store ACC into addr 12)
10: 7            (data)
11: 5            (data)
12: ?            (result destination)

```

Cycle 1 LOAD 10

```

FETCH   MAR ← [PC]=0      PC ← 1
        MDR ← Memory[0]="LOAD 10"
        CIR ← [MDR]
DECODE  opcode=LOAD, operand=10
EXECUTE MAR ← 10
        MDR ← Memory[10]=7
        ACC ← [MDR]=7

```

Cycle 2 ADD 11

```

FETCH   MAR ← [PC]=1      PC ← 2
        MDR ← Memory[1]="ADD 11"
        CIR ← [MDR]
DECODE  opcode=ADD, operand=11
EXECUTE MAR ← 11
        MDR ← Memory[11]=5
        ALU: ACC ← [ACC]+[MDR] = 7+5 = 12
        SR: Z=0, N=0, C=0, V=0

```

Cycle 3 STORE 12

```

FETCH   MAR ← [PC]=2      PC ← 3
        MDR ← Memory[2]="STORE 12"
        CIR ← [MDR]
DECODE  opcode=STORE, operand=12
EXECUTE MAR ← 12
        MDR ← [ACC]=12
        Memory[12] ← [MDR] (CU asserts WRITE on control bus)

```

Factors Affecting Processor Performance

Four factors are named explicitly in the H446 specification: **clock speed, number of cores, cache, and pipelining.**

Clock Speed

The clock is an electronic oscillator generating a square wave at a fixed frequency measured in **hertz**. Each pulse drives one or more stages of an instruction. Doubling the clock speed approximately doubles the number of cycles per second, which increases throughput for sequential, single-threaded code.

- **Limits:** heat output scales roughly with frequency³ (dynamic power $\propto C \cdot V^2 \cdot f$, and V must rise with f). Above ~5 GHz the chip cannot be cooled with air.
- **Overclocking** raises the clock manually at the cost of heat, power and chip lifespan.

Number of Cores

A **core** is an independent processor with its own ALU, CU and registers on a single chip. n cores can execute up to n threads truly simultaneously (true parallelism, not just concurrency).

- Useful for multi-threaded software (video encoders, web servers, compilers, games) and multi-tasking.
- **Amdahl's Law:** speed-up is limited by the *sequential* portion of the program. If 20% of a task is inherently sequential, maximum speed-up is $1 / 0.2 = 5\times$ no matter how many cores.
- Overhead: synchronisation, locks, and cache coherence between cores eats into the gain.

Cache

Fast on-chip SRAM that stores copies of recently used instructions and data. Drastically reduces **average** memory access time.

Level	Typical size	Access time	Shared?	Notes
Registers	~1 KB total	< 1 cycle	Per core	Referenced for comparison — fastest possible storage.
L1	32–64 KB per core (usually split i/d)	~1–4 cycles	Per core	Fastest cache; Harvard-style split instruction/data.
L2	256 KB–1 MB per core	~10–12 cycles	Per core (usually)	Larger, slightly slower; acts as a buffer for L1 misses.
L3	4–64 MB	~30–50 cycles	Shared across all	Victim cache of L2; also speeds inter-core data

			cores	sharing.
Main RAM	8–64 GB	~150–300 cycles (~50–100 ns)	System-wide	For comparison — the cost of a cache <i>miss</i> .

Locality of reference is the empirical observation that programs tend to re-use the same memory locations, or locations near each other, over short time periods.

- **Temporal locality:** a location accessed now is likely to be accessed again soon (e.g. loop counter, accumulator).
- **Spatial locality:** a location accessed now means nearby locations are likely to be accessed soon (e.g. traversing an array, sequential instructions).

Caches exploit both: when a miss occurs, a whole **cache line** (typically 64 bytes) is loaded, not just the requested word.

A **cache hit** is when the requested data is already in cache (fast). A **cache miss** forces a slower lookup in the next level down (L1 miss → L2 → L3 → RAM). **Hit rate** = hits / total accesses; modern CPUs achieve >95% L1 hit rates on typical code.

Worked Performance Calculation — Cache Effectiveness

Given: L1 access = 2 ns L1 hit rate = 95%
 RAM access = 100 ns (on a miss the access falls through to RAM)

Average access time = $0.95 \times 2 + 0.05 \times 100$
 = 1.9 + 5.0
 = 6.9 ns per access

Without any cache, every access is 100 ns.
 Speed-up from L1 cache = $100 / 6.9 \approx 14.5\times$

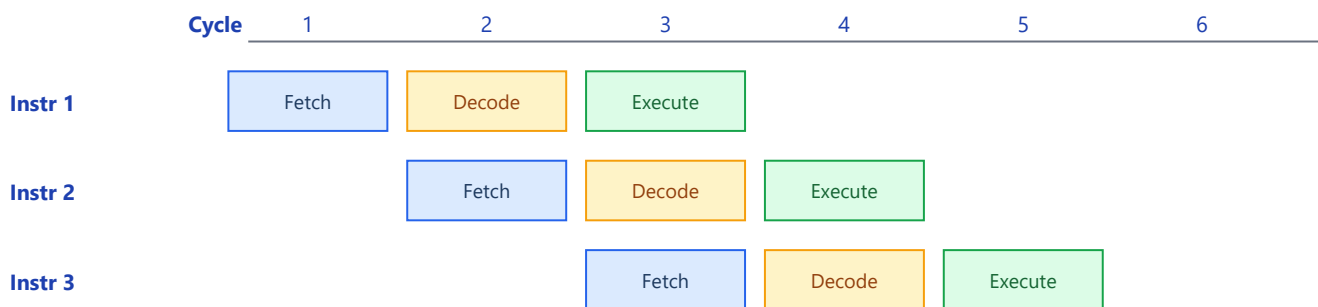
Observation: even though the miss penalty is 50× worse than a hit, the 95% hit rate means the average access time is dominated by the fast case. This is exactly why cache is so effective – combined with locality of reference, hit rates in the high 90s mean the slow RAM is rarely touched.

Pipelining

Pipelining is a performance technique where the FDE cycle is broken into stages, and multiple instructions are worked on *simultaneously* — each at a different stage. While instruction n is being executed, instruction $n+1$ is being decoded, and instruction $n+2$ is being fetched. It increases **throughput** (instructions completed per second) without increasing the clock speed.

In a simple 3-stage pipeline (Fetch / Decode / Execute), after the pipeline fills up, one instruction **completes every cycle** rather than every three cycles. Modern CPUs have much deeper pipelines (14–20+ stages).

3-Stage Pipeline — Timing Diagram



After cycle 3 the pipeline is full:

one instruction completes every cycle, not every 3 cycles.

Non-pipelined: 3 instrs $\times$ 3 cycles = 9 cycles.

Pipelined: 3 instrs = 5 cycles — 1.8 $\times$ speed-up.

Pipeline Hazards

Pipelines break down when an instruction cannot proceed because of its relationship with another instruction in flight. These are called **hazards**.

Hazard	Cause	Example	Mitigation
Data hazard	Instruction needs a result not yet produced by an earlier one still in the pipeline	ADD R1,R2,R3 ; SUB R4,R1,R5 — SUB needs R1 before ADD finishes writing it	Forwarding (feed ALU output directly to next ALU input) or pipeline stall (bubble)

Control hazard (branch)	Conditional branch — next instruction's address isn't known until execute completes	BEQ loop — should the pipeline fetch the instruction after BEQ or at loop ?	Branch prediction: guess the outcome and speculatively fetch. If wrong, flush the pipeline.
Structural hazard	Two instructions need the same hardware at the same time	A Von Neumann single-memory machine fetching an instruction while another instruction reads data	Duplicate hardware (split cache, Harvard-style) or stall one instruction

Branch prediction is a technique where the CPU guesses whether a conditional branch will be taken and speculatively fetches from the predicted address. **Static prediction** always predicts the same way (e.g. backward branches taken — usually correct for loops). **Dynamic prediction** uses a history table recording past outcomes of each branch. Modern predictors exceed 95% accuracy. A wrong prediction means the speculatively fetched instructions must be discarded (pipeline flush), wasting cycles.

Exam Tip: When asked "why does pipelining improve performance?", do not just say "it goes faster". State it precisely: "*Pipelining increases instruction throughput by overlapping the fetch, decode and execute stages of consecutive instructions so that one instruction completes per clock cycle once the pipeline is full, rather than once every three cycles.*" Then mention a hazard (e.g. branch) to show depth.

1.1.2 Types of Processor

CISC vs RISC

CISC (Complex Instruction Set Computer) uses a large set of complex, variable-length instructions, each of which may perform several low-level operations (e.g. load-modify-store in

one instruction). Originated when memory was scarce and expensive. Example: **x86 / x86-64** (Intel, AMD).

RISC (Reduced Instruction Set Computer) uses a small set of simple, fixed-length instructions, each executing in ideally one clock cycle. Only **load** and **store** instructions access memory (load-store architecture); everything else works register-to-register. Example: **ARM** (phones, Raspberry Pi, Apple M-series), **RISC-V**, **MIPS**.

Feature	CISC (e.g. x86)	RISC (e.g. ARM)
Instruction set size	Large (1000+ instructions)	Small (<100 core instructions)
Instruction length	Variable (1–15 bytes on x86)	Fixed (usually 4 bytes)
Cycles per instruction (CPI)	Variable, often many	Usually 1 (ideally)
Instruction complexity	High — can perform memory access + arithmetic in one instr	Low — each instruction does one simple thing
Registers	Fewer (historically 8–16 GPRs)	Many (16–32 GPRs)
Memory access	Most instructions can access memory directly	Only dedicated LOAD / STORE — load-store architecture
Code size	Smaller (dense instructions)	Larger (more instructions needed for same task)
Pipelining friendliness	Harder (variable length, complex decoding)	Easier (fixed length, uniform — ideal for deep pipelines)
Power consumption	Higher (more complex hardware)	Lower (simpler hardware) — dominant in mobile
Decode hardware	Complex (often microcoded)	Simple (hardwired)

Typical use	Desktops, laptops, servers	Phones, tablets, embedded, IoT, Apple Silicon Macs
-------------	----------------------------	---

Modern reality: the lines are blurred. Modern x86 CPUs internally translate CISC instructions into simpler RISC-like micro-operations (μ ops) which are then pipelined. ARM has added complex SIMD and cryptography instructions. The categories remain conceptually useful but no current high-performance CPU is purely one or the other.

Exam Tip: When asked to compare CISC and RISC, do not just list features — **justify the trade-off**. CISC makes each instruction do more, which saves memory and simplifies compiler work at the cost of complex decode hardware and harder pipelining. RISC makes the hardware simpler and more pipelineable at the cost of larger code size and more work for the compiler. Naming real examples (x86 / ARM) earns marks.

Worked Exam Question — CISC vs RISC

Q: "A smartphone manufacturer chooses a RISC processor over a CISC processor for their latest device. Justify this decision." [6 marks]

Model answer:

- (1) RISC instructions are simple and fixed-length, each ideally executing in one clock cycle. This makes the pipeline easier to keep full and allows deeper pipelines – higher instruction throughput per GHz.
- (2) RISC has simpler decode hardware (hardwired, not microcoded). This reduces the number of transistors on the chip, which directly reduces dynamic power consumption ($P \propto C \cdot V^2 \cdot f$). Lower power = longer battery life and less heat, both critical for a phone.
- (3) RISC architectures (e.g. ARM) provide many general-purpose registers, so compilers can keep intermediate values on-chip rather than in RAM. This reduces energy-expensive memory accesses – again improving battery life.
- (4) The trade-off is larger code size: each RISC instruction does less work, so programs contain more instructions than their CISC equivalents. On a phone with gigabytes of RAM this is a minor concern

compared to power and heat.

Conclusion: RISC delivers better performance-per-watt, which is the decisive metric in a battery-powered handheld device.

Multicore and Parallel Systems

A **multicore processor** contains two or more independent processing cores on a single chip, each with its own ALU, CU, registers and usually private L1 and L2 caches, sharing L3 cache and the memory controller. Cores can execute different threads **truly in parallel**.

Related terms:

- **Multi-processor**: multiple physically separate CPU chips in one system (used in servers).
- **Simultaneous Multithreading (SMT / Hyper-Threading)**: a single core appears as 2 logical cores to the OS by duplicating register files and running two threads interleaved through the pipeline. Gives ~20–30% extra throughput, not 2×.
- **SIMD (Single Instruction, Multiple Data)**: one instruction operates on a vector of values simultaneously (e.g. x86 SSE/AVX, ARM NEON). The core principle behind GPUs.
- **MIMD (Multiple Instruction, Multiple Data)**: what general-purpose multicore CPUs are — each core can run different code on different data.

Parallel processing is only effective if the task can be decomposed into independent sub-tasks. Tasks with heavy data dependencies or sequential logic gain little. Overheads of synchronisation (locks, cache coherency, communication) reduce real-world speed-up below the theoretical maximum (Amdahl's Law).

GPU — Graphics Processing Unit

A **GPU** is a highly parallel processor originally designed for rendering graphics. Architecturally it is massively **SIMD**: **thousands** of simple cores (NVIDIA: "CUDA cores") execute the same instruction on different data items simultaneously. A modern GPU might have 5,000–16,000 cores vs a CPU's 8–32.

GPUs excel at **data-parallel** workloads — the same operation performed independently on huge quantities of data:

- **Graphics** — shading and texturing millions of pixels per frame.
- **Machine learning** — training neural networks, which are almost entirely large matrix multiplications (vastly accelerated by SIMD). This is why GPUs dominate AI.
- **Scientific simulation** — fluid dynamics, climate modelling, molecular dynamics.
- **Cryptography and cryptocurrency mining** — e.g. SHA-256 hashing.
- **Video encoding/decoding** — parallel processing of blocks within each frame.

GPUs are *poor* at tasks with lots of branching, sequential logic, or irregular memory access — the CPU remains far better at single-threaded work, operating system tasks and control logic.

Feature	CPU	GPU
Cores	Few (4–64), complex	Thousands, simple
Clock speed	High (3–5 GHz)	Lower (1–2 GHz)
Design philosophy	Low latency, complex control	High throughput, mass SIMD
Cache	Large, hierarchical (L1/L2/L3)	Small per core, large shared memory
Branch handling	Excellent (branch predictors)	Poor (cores run in lockstep)
Best at	Sequential logic, OS, varied tasks	Data-parallel maths, ML, graphics

Co-processors

A **co-processor** is a dedicated processor that assists the main CPU by performing a specialised class of operations more efficiently. The CPU offloads tasks to it, freeing the CPU for other work.

Examples:

- **FPU (Floating-Point Unit)** — handles floating-point arithmetic. Historically a separate chip (e.g. Intel 8087); now integrated inside modern CPUs.
- **GPU** — acts as a co-processor for graphics and SIMD data-parallel work.
- **DSP (Digital Signal Processor)** — specialised for audio, radio, modem signal processing; optimised for the multiply-accumulate operation.
- **Cryptographic co-processors** — e.g. AES-NI hardware on x86, TPM chip for key storage.
- **NPU / AI accelerator** — neural network co-processor in modern phones (Apple Neural Engine, Google Tensor).

1.1.3 Input, Output and Storage Devices

Classifying Devices

Category	Direction of data	Examples
Input	Into the computer	Keyboard, mouse, scanner, microphone, sensors
Output	Out of the computer	Monitor, printer, speakers, actuators
Storage	Both (read + write), persistent	HDD, SSD, optical disc, magnetic tape

Devices can also be classified by how humans interact with them:

- **Manual input:** a human deliberately provides data (keyboard, mouse, touchscreen).
- **Automatic input:** data is captured by sensors or readers without human keying — RFID tag at a warehouse, temperature sensor on a boiler, barcode on a product. Eliminates human transcription error and works continuously.

Input Devices

Device	How it works	Typical use
Keyboard	Matrix of switches; microcontroller scans the matrix, sends scan codes over USB/Bluetooth.	Text input, command entry

Mouse	Optical sensor (LED + tiny camera) tracks surface texture; changes sent as dx/dy deltas.	GUI pointing
Scanner	Strip of CCD/CIS sensors illuminated by LED; captures reflected light to produce a bitmap. OCR software may then convert the image to text.	Digitising documents and photos
Microphone	Diaphragm vibrates with sound waves, producing an analogue voltage; ADC samples at e.g. 44.1 kHz to produce digital audio.	Voice input, recording, calls
Barcode reader	Laser or CCD reads the widths of black/white stripes representing a number; software looks the code up in a database.	Retail, inventory, libraries
RFID reader	Reader emits a radio field; a passive tag (with no battery) uses that energy to transmit its unique ID back. Active tags have batteries and longer range.	Contactless cards, warehouse tracking, passports, livestock tagging
Biometric sensors	Capture a physical or behavioural characteristic: fingerprint (capacitive/optical), iris pattern, face geometry, voice. Converted to a template and compared to a stored one.	Authentication, border control, phone unlock
Sensors (automatic input)	Transducers convert a physical quantity to an electrical signal; ADC digitises it. Examples: temperature (thermistor, thermocouple), pressure (piezoelectric), light (LDR, photodiode), humidity , proximity , accelerometer/gyro .	Embedded/control systems, IoT, cars, weather stations, factory automation
Touchscreen	Capacitive: detects change in electric field when a finger touches the glass. Resistive: two conductive layers pressed together.	Phones, kiosks, tablets

Output Devices

Monitors

Type	How it works	Pros / cons
LCD (Liquid Crystal Display)	A fluorescent backlight (historically CCFL) shines through a sheet of liquid crystal cells. Each cell twists light by an amount controlled by voltage, and a colour filter and polariser determine which colours reach the viewer.	Thin, cheap, energy-efficient vs CRT; but requires separate backlight and has limited contrast (blacks are grey).
LED (LED-backlit LCD)	Same LCD panel, but backlit with LEDs instead of a fluorescent tube. The LCD still forms the image; "LED TV" is a marketing term for LED-backlit LCD.	Thinner, more energy-efficient, better contrast than CCFL (local dimming possible), longer lifetime.
OLED (Organic LED)	Each pixel is a tiny organic LED that emits its own light when current is applied — no backlight . Black pixels are completely off.	Perfect blacks, infinite contrast, very thin, flexible, wide viewing angle. Drawbacks: more expensive, organic materials degrade (especially blue pixels) causing possible burn-in .

Printers

Type	How it works	Typical use
Inkjet	Print head sprays microscopic droplets of liquid ink onto paper. Droplets are formed by thermal (heating ink to create a bubble) or piezoelectric methods.	Home printing, photos. Cheap hardware, expensive ink.
Laser	Laser draws the page image as an electrostatic charge pattern on a rotating drum. Charged toner (powdered plastic/pigment) sticks to those areas, is transferred to paper, and fused by heat/pressure from the fuser unit.	Office, high-volume text. Fast, high quality, lower cost per page.

3D printer	Additive manufacturing — builds an object layer by layer from a digital model. FDM : extrudes molten plastic filament. SLA : uses a UV laser to cure liquid resin. SLS : sintered powder with a laser.	Prototyping, custom parts, medical implants, small-run manufacturing.
-------------------	---	---

Speakers and Actuators

- **Speakers**: DAC converts digital audio to an analogue voltage, amplified and sent to a coil. The coil in a magnetic field moves a diaphragm, producing sound waves.
- **Actuators** are output devices used in control systems to *physically act on the environment*: motors (rotation), solenoids (linear push/pull), servo motors (precise angular position), relays (switch high-current circuits), heaters, valves, speakers. Essential in embedded/robotic systems — the output that closes the feedback loop started by sensors.

Storage Devices

Primary Storage

Primary storage is memory the CPU can access *directly* via the memory bus: RAM, ROM, cache and registers. Characterised by fast access and relatively small capacity.

Type	Technology	Volatile?	Properties	Use
DRAM (Dynamic RAM)	Each bit stored as charge on a tiny capacitor with one transistor per cell (1T1C). Charge leaks and must be refreshed every few milliseconds.	Volatile	High density, low cost per bit, relatively slow. Needs refresh circuitry.	Main memory (RAM) — GB-scale capacity.
SRAM (Static RAM)	Each bit stored in a flip-flop made from 6 transistors. Holds state as long as power is applied — no refresh.	Volatile	Very fast, very expensive per bit, lower density than DRAM.	CPU caches (L1, L2, L3).

ROM (Read-Only Memory)	Contents fixed at manufacture or programmed permanently (PROM / EPROM / EEPROM / flash variants).	Non-volatile	Read only (or rarely written), retains contents without power.	Boot firmware (BIOS/UEFI), embedded system program code.
----------------------------------	---	--------------	--	--

Volatile vs Non-Volatile

- **Volatile:** contents are lost when power is removed (RAM, cache, registers).
- **Non-volatile:** contents are retained without power (ROM, HDD, SSD, optical, tape).

Secondary Storage

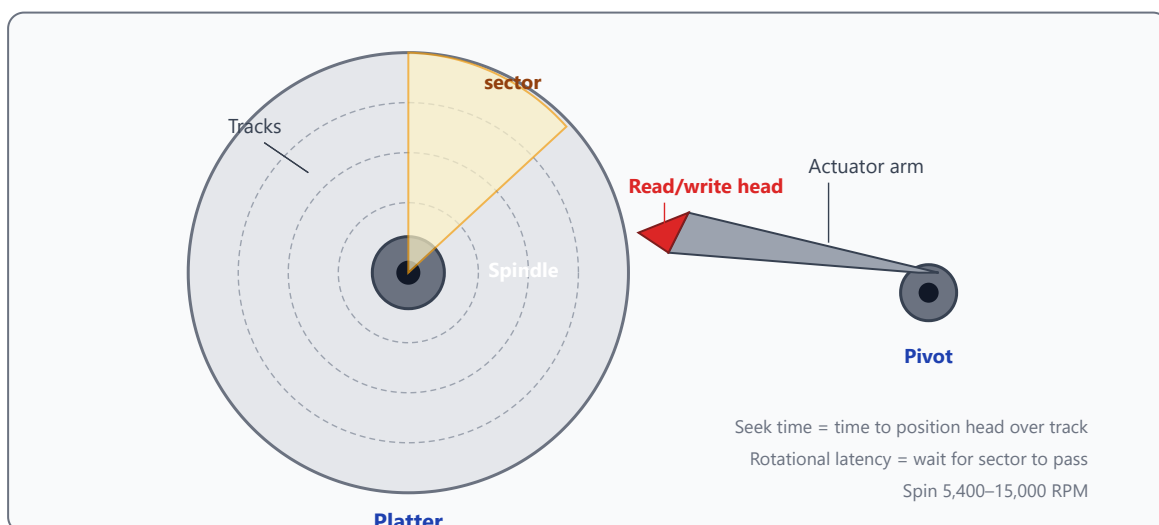
Secondary storage is non-volatile storage the CPU accesses indirectly (via I/O controllers).

Data must be loaded into RAM before the CPU can process it. Used for long-term retention of OS, applications and user data.

HDD (Hard Disk Drive)

Stores data as **magnetic patterns** on the surface of spinning aluminium or glass **platters** coated with a thin ferromagnetic film. A **read/write head** floats nanometres above the platter, detecting (read) or altering (write) the magnetisation of tiny regions. Data is organised in concentric **tracks**, subdivided into **sectors** (typically 512 B or 4 KB). The **spindle** spins platters at 5,400–15,000 RPM; the **actuator arm** positions the head over the correct track.

HDD Internal Structure



- **Pros:** very high capacity (1–20 TB), extremely cheap per GB.
- **Cons:** mechanical parts → slow (~5–10 ms seek), fragile (shock-sensitive), noisy, higher power.

SSD (Solid State Drive)

Stores data in **NAND flash memory** — grids of floating-gate transistors where trapped electrical charge represents a bit (or multiple bits in MLC/TLC/QLC cells). A controller manages reads, writes and **wear levelling** (spreading writes across all cells since each cell can only be erased a finite number of times — typically 1,000–100,000).

- **Pros:** no moving parts → near-instant access (<0.1 ms), silent, shock-resistant, low power. NVMe SSDs reach 7,000+ MB/s.
- **Cons:** more expensive per GB than HDD; **finite write endurance** (TBW rating); data retention without power degrades slowly over years.

Optical Discs (CD / DVD / Blu-ray)

Data is stored as microscopic **pits** and **lands** on a reflective spiral track. A laser shines onto the track: light reflects differently from pits and lands, and the reflected intensity is detected as binary data.

- **CD:** infrared laser (780 nm), ~700 MB.
- **DVD:** red laser (650 nm) — shorter wavelength allows smaller pits, ~4.7 GB (single-layer).
- **Blu-ray:** blue-violet laser (405 nm) — even smaller pits, 25 GB single-layer, 50 GB dual-layer.
- Writable: R discs use a heat-sensitive dye that the laser burns (permanent); RW discs use a phase-change alloy that can be reset.

Magnetic Tape

Thin plastic tape coated in ferromagnetic material, wound in a cartridge. A head magnetises tracks along the tape as it winds from one reel to another. LTO-9 cartridges store up to 18 TB native (45 TB compressed).

- **Sequential access only** — random access requires winding the tape to the correct position (can take minutes).
- Cheapest storage per GB; used for long-term archival (broadcast archives, cold cloud storage, disaster recovery).

Secondary Storage Comparison

Type	Capacity	Speed	Cost/GB	Durability	Typical use
HDD	1–20 TB	~100–200 MB/s, 5–10 ms seek	Very low (~£0.02/GB)	Fragile (shock); magnetic patterns stable for decades if undisturbed	Bulk desktop / NAS / servers
SSD	128 GB–8 TB	500–7,000 MB/s, <0.1 ms	Medium (~£0.08/GB)	Shock-resistant; finite write cycles	OS drive, laptops, gaming, performance-critical
Optical	700 MB (CD) – 50 GB (BD)	Slow (~1–72 MB/s)	Very low per disc	Scratch-sensitive; 50–100 yr if kept properly	Distribution, archival, read-only media
Magnetic tape	Up to 18 TB/cartridge	Fast sequential (~400 MB/s); very slow random	Extremely low (~£0.003/GB)	Excellent 30+ year shelf life if stored properly	Long-term archival, cold backup

Choosing the Right Storage — Worked Scenarios

Scenario 1: A data centre provider stores legal archive data that is almost never read but must be retained for 10+ years.

Answer: Magnetic tape. Cheapest cost/GB by orders of magnitude; sequential-only access is acceptable because data is rarely retrieved; excellent shelf life if stored correctly.

Scenario 2: A YouTuber edits 4K video on a laptop and needs fast project loading and scrubbing.

Answer: NVMe SSD. Random 4K read/write speed matters for video

scrubbing; no moving parts suits a portable device; the higher cost/GB is acceptable at moderate capacities.

Scenario 3: A software company ships a boxed consumer product to high-street retailers.

Answer: Optical disc (DVD or Blu-ray). Very cheap per unit, universally readable, tamper-evident, easy to distribute physically. Read-only media suits the use case.

Scenario 4: A photographer wants to store 50 TB of RAW photos at home with the best cost/GB.

Answer: HDD (ideally in a RAID NAS). Capacities of 16–20 TB per drive are cheap; the photographer rarely needs millisecond random access so the HDD's speed limitation is not a problem.

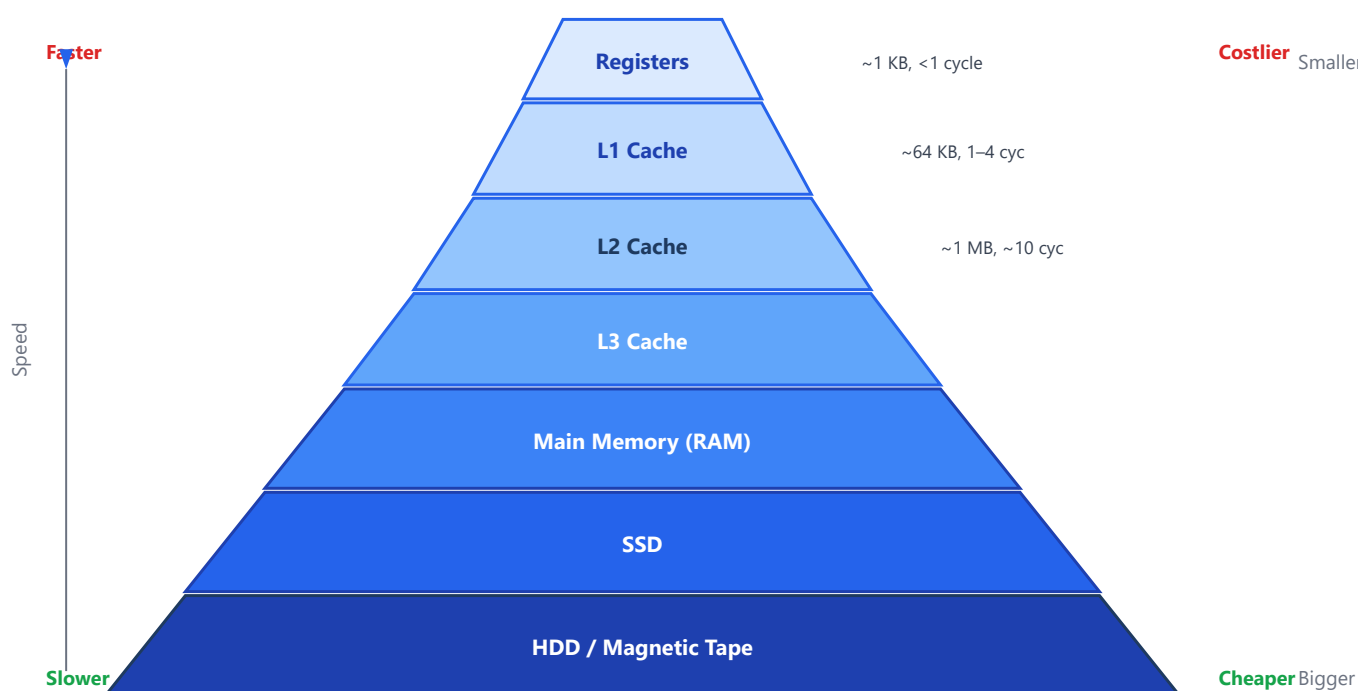
Scenario 5: A car's engine ECU needs to store its firmware.

Answer: Flash / ROM. Non-volatile so firmware survives power-off; small capacity but very reliable; no moving parts handles the engine bay's vibration environment.

Memory Hierarchy

Storage forms a hierarchy, trading speed for capacity and cost. The CPU first looks in registers, then L1, L2, L3, RAM, and only finally in secondary storage if necessary.

The Memory Hierarchy



Each level acts as a cache for the level below it

RAM vs ROM vs Flash

Property	RAM (DRAM)	ROM	Flash
Volatile?	Yes	No	No
Read/write?	Read and write freely	Read only (or rarely rewriteable)	Read freely; write in blocks, limited cycles
Typical use	Working memory for the OS and running programs	Firmware (BIOS/UEFI); boot code	SSDs, USB sticks, SD cards, phone storage, embedded firmware
Speed	Fast	Slow (often read-only lookups)	Medium (faster read than write)
Endurance	Unlimited writes	Fixed at manufacture or limited	Limited erase/write cycles (wear)

Example	16 GB DDR4/DDR5 stick	UEFI firmware chip on motherboard	NVMe SSD, SD card, eMMC on a phone
---------	-----------------------	-----------------------------------	------------------------------------

Exam Tip: A very common question: "*Why does a computer need both RAM and ROM?*" Answer: **ROM is non-volatile** and holds the bootstrap firmware needed immediately on power-on — before any OS is loaded. **RAM is volatile** but fast and rewritable, providing working memory for the OS, applications and data once running. You cannot replace RAM with ROM (ROM is slower and not writable for working data) or ROM with RAM (RAM loses its contents at power-off, so the boot code would be gone).

Virtual Storage

Virtual storage is storage that *appears* local to the user or system but is physically provided by one or more remote/distributed servers, typically over a network. Examples: Google Drive, OneDrive, Dropbox, iCloud, Amazon S3, network-attached storage (NAS), SAN (Storage Area Network). The storage hardware is abstracted away behind a uniform interface.

Characteristics and benefits:

- **Scalable** — capacity can grow on demand without changing local hardware.
- **Access from anywhere** — the same files are available on multiple devices over the internet.
- **Pooled and abstracted** — multiple physical devices (HDDs, SSDs, tapes) are presented as one logical volume; the user does not see which physical disk their data lives on.
- **Redundancy** — providers replicate data across drives / data centres, improving durability and availability.
- **Offsite backup** — protects against local disasters (fire, theft, flood).

Drawbacks:

- Requires a reliable **internet connection** — offline access is limited.
- Ongoing **subscription cost** rather than one-off hardware purchase.
- **Security / privacy** concerns — data is held on third-party servers (encryption and trust become critical).

- **Latency** — slower than local SSD for large transfers.

Exam Tip: "Virtual storage" in H446 means the *abstraction* of storage over a network or pool — do not confuse it with **virtual memory** (the OS technique of using a page file on disk to extend apparent RAM). Virtual memory is a 1.2.1 topic; virtual storage is a 1.1.3 topic.

Key Definitions — Quick Reference

Throughput — the number of instructions completed per unit time. Pipelining and multiple cores improve throughput; clock speed improves both throughput and per-instruction latency.

Latency — the time from starting a single operation to its completion. A deep pipeline may *increase* the latency of one instruction (it passes through more stages) while increasing overall throughput.

Clock cycle — one tick of the CPU's clock. Each cycle one (or more) micro-operations happen. A 3 GHz CPU has a cycle time of $1/(3 \times 10^9) \text{ s} = \sim 0.33 \text{ ns}$.

Cache line — the unit of data moved between cache and memory on a miss. Typically 64 bytes. Loading a whole line at once is how hardware exploits spatial locality.

Word size — the natural unit of data handled by a CPU (e.g. 64-bit CPUs operate on 64-bit words). Equal to the size of most general-purpose registers and typically the data bus width.

Parallelism — multiple computations happening at literally the same instant (requires multiple cores, SIMD units, or GPUs). Distinct from **concurrency** (multiple tasks in progress, possibly time-sliced on one core).

Exam Technique — Summary Checklist

- Use **register transfer notation** for FDE questions: $MAR \leftarrow [PC]$, not prose.
- Always state the PC increment as a separate step during fetch.
- Distinguish clearly between **MAR (address)** and **MDR (data)**.
- When comparing CISC vs RISC, quote **examples** (x86, ARM) and explain the **trade-off**, not just features.
- Name the **Von Neumann bottleneck** in architecture-comparison questions.
- For pipelining, mention that throughput improves because stages overlap, and cite **a named hazard** (data / control / structural) for depth.
- For cache, mention **locality of reference** (temporal + spatial) and **hit/miss**.
- For storage recommendations, justify using a property of the technology: moving parts vs none, sequential vs random access, cost/GB vs speed.
- For GPU questions, emphasise **SIMD** and **thousands of simple cores**, and mention non-graphics uses (ML, simulation, crypto).
- For virtual storage, do not confuse it with virtual memory.